

■■■: LCEL -- LangChain ■■■■■

■■■: Chain -- ■■■■■■

■■■: Memory -- ■■■■■■

```

graph LR
    Memory --> LangChain
    LangChain --> RAG
    RAG --> ConversationBufferMemory
    ConversationBufferMemory --> Prompt
    Prompt --> ConversationSummaryMemory
    ConversationSummaryMemory --> LLM
    LLM --> token
    Memory --> ConversationBufferMemory
  
```

```
graph.invoke()
AgenticRAGState chat_history generate_node
```

Tool Calling -- LLM

```
Tool Calling ( ) LLM Tool Calling LLM
API
```

```
LangChain @tool StructuredTool ( LLM
) schema( LLM )
```

```
LLM Tool Calling -- ( )
LLM LLM - - Agent
```

:

```
LLM -- JSON Markdown
(Output Parser)
```

```
StrOutputParser -- JsonOutputParser LLM JSON
Pydantic schema Self Query Retriever LangChain
LLM
```

```
from langchain_core.output_parsers import PydanticOutputParser from pydantic import
BaseModel, Field class PersonInfo(BaseModel): name: str = Field(description="") age: int =
Field(description="") profession: str = Field(description="") parser =
PydanticOutputParser(pydantic_object=PersonInfo)
```